Reviewer Pack — Minimal Rank of Kneser Graphs Bounds Monotone Circuit Size

Entry 0a2cc1665148 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 15:55:11 UTC

Minimal Rank of Kneser Graphs Bounds Monotone Circuit Size

Entry ID: `0a2cc1665148` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-28 15:55:11 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Geometry (Kneser Graphs) **Field B** (complexity object): Complexity Theory: Boolean Function Complexity

Statement:

> {'sentence_1': 'For any given n, there exists a constant c(n) such that for every monotone boolean function f with n variables, the Kneser graph K(c(n), n/2) has a vertex set V and an edge set E such that |V| = n and |E| ≤ 2^n - c(n).', 'sentence_2': 'The minimal rank of the boolean function f is bounded by the size of its representation on Kneser graph, i.e., min Rank_E(f) ≤ |E|.', 'sentence_3': 'If a monotone boolean function f cannot be represented with fewer than 2^n - c(n) edges on any Kneser graph K(c(n), n/2), then it requires a circuit of size at least 2^(n/2).'}

Rationale (proposer's reasoning):

> {'sentence_1': 'Kneser graphs are interesting combinatorial structures that have been studied in combinatorics and geometric group theory, but not extensively applied to complexity theory.', 'sentence_2': 'Using the properties of Kneser graphs might reveal new insights into the structure of boolean functions and their representation by circuits.', 'sentence_3': 'A constructive mapping from a boolean function to its representation on a Kneser graph could potentially lead to faster algorithms for circuit lower bounds.'}

Taxonomy category: `COMBINatorial_Geometry × Complexity_Theory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `01ed274eb8a35477`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all n ≤ 40, every monotone boolean function f maps to a Kneser graph K(c(n), n/2) with |E| ≤ 2^n - c(n) and min Rank_E(f) ≤ |E|; it is falsified if there exists an n ≤ 40 such that for some monotone boolean function f, the mapping results in min Rank_E(f) > |E| or |E| > 2^n - c(n).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Kneser graph' AND 'monotone boolean function' AND 'circuit complexity' - 'vertex set' 'edge set' Kneser graph 'boolean function complexity' - min Rank_E('boolean function') 'size representation' Kneser graph

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_multiply(A, B):
    m, n = len(A), len(B[0])
    p = len(B)
    C = [[0 for _ in range(n)] for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gaussian_elimination(A, b):
    n = len(A)
    M = [A[i] + [b[i]] for i in range(n)]
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(M[j][i]) > abs(M[max_row][i]):
                max_row = j
```

```
        M[i], M[max_row] = M[max_row], M[i]
        factor = M[i][i]
        for j in range(n):
            M[i][j] /= factor
        for j in range(n):
            if i != j:
                factor = M[j][i]
                for k in range(n+1):
                    M[j][k] -= factor * M[i][k]
    return [row[-1] for row in M]

def min_rank(graph):
    V, E = graph
    n = len(V)
    max_edges = 0
    for subset_size in range(1, n//2 + 1):
        for subset in itertools.combinations(range(n), subset_size):
            subgraph = [e for e in E if all((i in subset and j not in subset) or (j in subset and
            max_edges = max(max_edges, len(subgraph))
    return max_edges

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.choice([5, 10, 15, 20, 30, 40])
    c_n = random.randint(1, n//2)
    V = list(range(n))
    E = []

    for i in range(n):
        for j in range(i+1, n):
            if abs(i - j) > c_n:
                E.append((i, j))

    rank = min_rank((V, E))
    metric_value = len(E)
    instances_tested = 1
    conjecture_holds = rank <= len(E)
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "min_rank",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 2

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
```

```
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(re
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b8152aff.py", line 98, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b8152aff.py", line 78, in run_trial
    rank = min_rank((V, E))
           ^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b8152aff.py", line 61, in min_rank
    subgraph = [e for e in E if all((i in subset and j not in subset) or (j in subset and i not in subset) 
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b8152aff.py", line 61, in <genexpr>
    subgraph = [e for e in E if all((i in subset and j not in subset) or (j in subset and i not in subset) 
                                                                          ^^^^
TypeError: cannot unpack non-iterable int object
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means we cannot verify the conjecture's conditions for n ≤ 40. | next: Re-run the test with a different seed or on a different system to ensure it completes successfully and provides results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13999
2	preregistration	ollama_remote	glm4:latest	0	0	6258
3	novelty	ollama_remote	glm4:latest	0	0	4806
4	novelty	ollama_remote	glm4:latest	0	0	5249
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14175
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11572
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9715
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12446
9	judge	ollama_remote	glm4:latest	0	0	14011

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 92232 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0a2cc1665148.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0a2cc1665148.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0a2cc1665148.tar.gz` (if generated)